SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM080
Name	Ratan Ranjeet Sankpal
Division	B
Date	11 October 2026
GitHub / Portfolio Link (optional)	_____

PART A: Architecture Decision Record (ADR)

Title: Use Depth-First Search (stack / recursion) as the default search strategy inside the AI Agent, with BFS kept as a swappable alternative

Status: Accepted

Date: 11 October 2026

1. Context

My Tic Tac Toe AI System (SLE-1) lets a human play against an AI Agent that searches the game tree without any heuristic. Two uninformed search orders were possible for the agent: BFS (queue, level by level) and DFS (stack/recursion, deep first). In SLE-2 I profiled both on the same 3×3 game with cProfile, time.perf_counter() and node counters, on boards with 3, 5 and 9 empty cells. In SLE-3 I placed both inside one AI Agent component that shares a Successor Generator and Goal Test and differs only in the Frontier. I now had to decide which strategy the system should use by default when it picks the computer's move.

2. Decision

The AI Agent uses DFS (recursive search with bottom-up MAX/MIN value backup and undo_move) as its default strategy. BFS stays in the code behind the same Frontier interface (queue instead of stack), so it can be selected through best_move(board, method) for comparison or teaching.

3. Alternatives Considered

- **Alternative 1: BFS as the default.** Explores level by level and exposes the shortest winning sequence, but builds and stores the whole generated tree before backing up values.
- **Alternative 2: Iterative Deepening DFS.** Keeps DFS-like memory but explores level by level. Rejected because it repeats work at every depth, and my system needs the whole tree for value backup anyway. I did not measure it in SLE-2.
- **Alternative 3: Informed search (Greedy / A*) with a heuristic.** Rejected because this system is deliberately heuristic-free and a Tic Tac Toe board gives no natural distance-to-goal measure.

4. Rationale / Justification

Evidence from my SLE-2 profiling (same positions, same implementation, five runs each):

Test case	Algo	Avg. time (ms)	Nodes expanded	Move
Best (3 empty cells)	DFS	0.0083	4	Cell 8
Best (3 empty cells)	BFS	0.0217	5	Cell 8
Average (5 empty cells)	DFS	0.2060	209	Cell 8
Average (5 empty cells)	BFS	0.3730	210	Cell 8
Worst (9 empty cells)	DFS	458.5446	549,945	Cell 0
Worst (9 empty cells)	BFS	1750.5920	549,946	Cell 0

- **Same work, different speed.** Both algorithms visited essentially the same number of states (549,945 vs 549,946 on the empty board), so BFS is slower not because it explores more, but because of how it traverses and stores them.

- **DFS was faster in every case and the gap grew with problem size:** about 2.6× (best), 1.8× (average) and 3.8× (worst: 458.5 ms vs 1750.6 ms).
- **Profiling graph (Fig. 2, SLE-2):** DFS time sits in the recursive minimax_dfs path, which stays shallow (a board has at most 9 moves). BFS time goes into expand_level()/queue.pop() across 549,946 nodes and a full generated tree kept for the later backup.
- **Memory.** DFS holds only the current path (at most 9 boards deep). Counting the levels of the same game tree, the widest level has 200,448 boards, which BFS has to keep in its frontier.
- **No loss of quality.** Both returned the same move in all three test cases (Cell 8, Cell 8, Cell 0), so choosing DFS does not change what the AI plays.
- **Fits my C4 design (SLE-3).** The Frontier is the only component that differs (Queue = BFS, Stack = DFS), so making DFS the default needs no change to the Game Board, Game Controller or Goal Test.

5. Consequences

Positive:

- Fastest option I measured: 458.5 ms for the full empty-board search, instant for mid-game positions (0.2 ms).
- Very small memory use (depth ≤ 9) compared with a BFS frontier.
- BFS remains available, so the SLE-2 comparison can be re-run at any time with one switch.

Negative / Trade-offs:

- DFS still searches the whole tree (about 550k nodes). This is fine for 3×3 but will not scale to Connect Four, which would need a depth limit, a heuristic or a visited set for repeated boards.
- Unlike BFS, DFS does not by itself give the shortest winning sequence, so among equally good moves it may choose a slower win.
- Timings are machine- and Python-specific (as noted in SLE-2); the ranking is reliable, the exact milliseconds are not universal.

6. References (optional)

SLE-2 Profiling Report: Section 4 (results table) and Fig. 2 (py-spy graph). SLE-3 Architectural Design: Level 2 container diagram, Level 3 component diagram (Frontier) and Section 6 (Design Decisions).

PART B: Short Summary of My SLE Journey

SLE-1: I built the Tic Tac Toe AI with BFS and DFS search, plus an AI contribution log and README. SLE-2: I profiled both versions on 3-, 5- and 9-empty-cell boards; the node counts were almost equal, but DFS ran 1.8× to 3.8× faster (458.5 ms vs 1750.6 ms on the empty board) and used less memory. SLE-3: I drew the C4 model and kept BFS and DFS inside one AI Agent where only the Frontier differs. This ADR connects them: the SLE-2 numbers and the SLE-3 design together justify choosing DFS as the default strategy.

PART C: AI Contribution Note

- **AI tools used (if any):** Claude (Anthropic).
- **What AI helped with:** drafting the ADR structure and wording from my SLE-2 and SLE-3 reports, and suggesting viva questions. It also counted the level widths of the Tic Tac Toe game tree (the 200,448 figure) with a short script.
- **What I did myself:** wrote and ran the code (SLE-1), chose the test cases, ran the profiling and checked the measurements (SLE-2), designed the C4 diagrams (SLE-3), chose which decision to record, and checked every number in this ADR against my reports.

PART D: Viva Preparation Notes

- **Why DFS?** Same ~549,945 nodes as BFS but faster in all cases (3.8× on the empty board) and tiny memory.

- **Why was BFS slower?** It generates and keeps the whole tree/frontier (widest level 200,448 boards) before backing up values; DFS keeps only a path of ≤ 9 boards.
- **1-minute container tour:** Input → Game Controller ↔ AI Agent (BFS/DFS), Game Board read/updated by the controller, Output prints the result.
- **If the board gets bigger:** full search is not feasible; add a depth limit with a heuristic, or a visited set for repeated boards.
- **SLE-2 vs SLE-3 wording:** SLE-2 measured full MAX/MIN value backup; SLE-3 describes the same successor generator and goal test as searching for an AI win. Be ready to explain they are the same search with the Frontier as the only difference.